

Programmierung 2 - Sommersemester 2026

Prof. Dr. Markus Esch

Übung Nr. 11 Abgabe KW 29

1. Aufgabe

Entwickeln Sie ein Programm in C, das eine Datenbank für Mitarbeiterinformationen mithilfe von Arrays und Structs verwaltet. Das Programm soll in der Lage sein, Daten hinzuzufügen, zu bearbeiten, zu löschen und abzufragen. Setzen Sie dabei folgendes um:

- i Definieren Sie ein struct namens `Mitarbeiter`, das die folgenden Informationen eines Mitarbeiters speichert:
 - Name (String)
 - Alter (Integer)
 - Abteilung (String)
 - Gehalt (Float)
- ii Initialisieren Sie ein Array von `Mitarbeiter`-Structs, das bis zu 50 Mitarbeiter speichern kann.
- iii Implementieren Sie die folgenden Funktionen:
 - `void addMitarbeiter()`: Diese Funktion fügt einen neuen Mitarbeiter zum Array hinzu. Der Benutzer soll aufgefordert werden, die Daten des Mitarbeiters über die Konsole einzugeben.
 - `void deleteMitarbeiter(char* name)`: Diese Funktion löscht einen Mitarbeiter aus dem Array basierend auf dem angegebenen Namen.
 - `void editMitarbeiter(char* name)`: Diese Funktion ermöglicht es, die Daten eines Mitarbeiters zu bearbeiten. Der Benutzer kann auswählen, welche Daten (z.B. Alter, Abteilung, Gehalt) aktualisiert werden sollen.
 - `void queryMitarbeiter(char* name)`: Diese Funktion sucht nach einem Mitarbeiter basierend auf dem Namen und gibt dessen Daten aus.
 - `void printAllMitarbeiter()`: Diese Funktion druckt die Informationen aller Mitarbeiter im Array.
- iv Erstellen Sie ein einfaches Konsolenprogramm, das es Benutzern ermöglicht, zwischen den verschiedenen Funktionen zu wählen (z.B. Mitarbeiter hinzufügen, löschen, bearbeiten, abfragen, alle anzeigen).
- v Fügen Sie Fehlerbehandlungen hinzu, zum Beispiel für den Fall, dass das Array voll ist oder ein nicht vorhandener Mitarbeiter bearbeitet oder gelöscht werden soll.

2. Aufgabe

- (a) Implementieren Sie in C eine Funktion `multiply`, welche zwei Pointer auf Integer-Werte als Argumente entgegennimmt und das Produkt der beiden Zahlen als Rückgabewert zurück gibt.
- (b) Implementieren Sie ein Testprogramm, um Ihre Funktion zu testen.

3. Aufgabe

Matrizen können in C als zweidimensionale Arrays dargestellt werden. Implementieren Sie eine Bibliothek, welche folgenden Funktionen für Integer-Matrizen bereitstellt:

i `void printMatrix(int matrix[], int rows, int cols)`: Gibt die Matrix formatiert aus. Geben Sie die Matrix in geeigneter Weise aus, z.B. folgendermaßen:

```
| 2 18 1 |
| 27 16 24 |
| 42 18 9 |
| 5 10 42 |
| 8 7 14 |
```

- ii `int findMaxInCol(int matrix[], int rows, int cols, int col)`: Findet das Maximum in einer bestimmten Spalte.
- iii `int findMinInRow(int* matrix, int rows, int cols, int row)`: Findet das Minimum in einer bestimmten Zeile.
- iv `void swapRows(int* matrix, int rows, int cols, int row1, int row2)`: Vertauscht zwei Zeilen der Matrix.
- v `void transposeMatrix(int* matrix, int rows, int cols)`: Transponiert die Matrix. Diese Funktion soll nur für quadratische Matrizen möglich sein.
- vi `void multiplyByScalar(int* matrix, int rows, int cols, int scalar)`: Multipliziert jedes Element der Matrix mit einem Skalar.
- vii `void sum(int* m1, int* m2, int* res, int rows, int cols)`: Addiert die Matrizen `m1` und `m2` und speichert das Ergebnis in `res`. Datenverarbeitung.
- viii `void mult(int* m1, int* m2, int* res, int m1_rows, int m1_cols, int m2_rows, int m2_cols)`: Multipliziert die Matrizen `m1` und `m2` und speichert das Ergebnis in `res`.

Die Funktionen sollen allgemein implementiert sein, sodass Arrays beliebiger Größe übergeben werden können.

Schreiben Sie ein Testprogramm, welches die Matrix-Funktionen testet.

Lernziele:

Nach der Bearbeitung des Übungsblattes sollten Sie ...

- ... Structs in C deklarieren und nutzen können, um komplexe Datenstrukturen effizient zu speichern und zu verwalten.
- ... Arrays in C nutzen können, um Sammlungen eines Datentyps zu speichern.
- ... in der Lage sein, Arrays in C zu deklarieren, zu initialisieren sowie deren Elemente zu lesen und zu schreiben.
- ... die Konsequenzen des fehlenden *Bounds Checking* bei Arrays in C erklären können.
- ... das Konzept der Pointer erklären und anwenden können.
- ... Pointer deklarieren, initialisieren, zuweisen und dereferenzieren können.
- ... Funktionen implementieren und aufrufen können, die Parameter über Pointer erhalten.

Zusatzaufgaben

Die folgenden Aufgaben sind Zusatzaufgaben und nicht Teil des Peer Reviews! Sie können die Aufgaben freiwillig bearbeiten, um die Inhalte weiter zu üben. Selbstverständlich können Sie auch Unterstützung und Beratung zu den Zusatzaufgaben erhalten.

1. Aufgabe

Bubblesort ist ein einfach zu implementierender Sortieralgorithmus. Der Pseudocode des Algorithmus sieht wie folgt aus:

```
procedure bubbleSort( A : list of sortable items ) defined as
    for each i in 1 to length(A) do:
        for each j in length(A) downto i + 1 do:
            if A[ j - 1 ] > A[ j ] then
                swap( A[ j - 1 ], A[ j ] )
            end if
        end for
    end for
end procedure
```

Implementieren Sie in C eine Funktion, die als Argument einen Pointer auf ein Integer-Array entgegennimmt und dieses mit Hilfe des Bubblesort-Algorithmus sortiert.

Implementieren Sie ein Testprogramm, um die Funktion zu testen.

2. Aufgabe

Es soll ein Programm zur Verwaltung des Autobestands in einem Autohaus entwickelt werden. Implementieren Sie dazu folgendes:

(a) Definieren Sie einen Typ `Auto` mit folgenden Eigenschaften:

- i Marke (der Markenname ist auf 20 Zeichen beschränkt)
- ii Höchstgeschwindigkeit
- iii Türenanzahl
- iv ABS (ja oder nein)
- v Sonderausstattung (eine Liste von Ausstattungsmerkmalen als Strings; ein Auto hat maximal 10 Ausstattungsmerkmale, jedes Ausstattungsmerkmal hat maximal 100 Zeichen)
- vi Motor Ein Motor soll durch einen eigenen Typ mit folgenden Eigenschaften dargestellt werden:
 - PS
 - Zylinder
 - Hubraum

(b) Erstellen Sie eine Funktion, welche die Eigenschaften eines Autos als Parameter entgegennimmt und ein `Auto` zurückliefert.

(c) Schreiben Sie eine Funktion, welche den Wert eines Autos folgendermaßen schätzt:

- Maximalgeschwindigkeit multipliziert mit 50

- multipliziert mit der Türenanzahl
 - plus 5000 EUR, wenn das Auto ABS hat
 - plus 3000 EUR für jede Sonderausstattung
 - ist die Marke Porsche, multipliziert mit 2
- (d) Implementieren Sie eine Print-Funktion, welche ein Auto mit allen seinen Eigenschaften ausgibt.
- (e) Verwalten Sie die Menge der Autos eines Autohauses in einem Array. Implementieren Sie dazu Funktionen, um Autos hinzuzufügen und zu entfernen.
- (f) Implementieren Sie ein interaktives Testprogramm.

3. Aufgabe

Implementieren Sie eine Bibliothek zur Verarbeitung von Strings. Die Bibliothek soll folgende Funktionen beinhalten:

- `strcmp`: Nimmt zwei Strings entgegen und vergleicht diese. Liefert <0 wenn der erste String kleiner ist, 0 wenn beide Strings gleich sind und sonst >0 .
- `strspn`: Nimmt zwei Strings entgegen und liefert die Anzahl der Zeichen am Anfang des ersten Strings, die sämtlich im zweiten String vorkommen.
- `strlen`: Liefert die Länge eines übergebenen Strings, ohne `'\0'`.
- `indexOf`: Nimmt einen String und ein einzelnes Zeichen entgegen und liefert den Index des ersten Vorkommens des Zeichens. Falls das Zeichen nicht vorkommt, wird -1 zurückgegeben.
- `lastIndexOf`: Nimmt einen String und ein einzelnes Zeichen entgegen und liefert den Index des letzten Vorkommens des Zeichens. Falls das Zeichen nicht vorkommt, wird -1 zurückgegeben.
- `replace`: Nimmt einen String und zwei einzelne Zeichen entgegen. Ersetzt alle Vorkommen des ersten Zeichens durch das Zweite.
- `toUpperCase`: Nimmt einen String entgegen und wandelt alle Buchstaben in Großbuchstaben um.
- `swap`: Nimmt zwei Strings entgegen und vertauscht diese.

Implementieren Sie auch ein interaktives Testprogramm für Ihre String-Bibliothek.

4. Aufgabe

Erweitern Sie die Aufgabe 1 wie folgt:

- Implementieren Sie die Möglichkeit, die Mitarbeiterdaten nach Gehalt oder Alter zu sortieren und auszugeben.
- Ersetzen Sie das Array durch eine dynamische Datenstruktur ihrer Wahl.

5. Aufgabe

Passen Sie ihr Testprogramm zur Aufgabe 3 so an, dass die Größe sowie die Werte für Matrizen vom Nutzer abgefragt werden.

Hinweis: Um diese Aufgabe lösen zu können, benötigen Sie dynamische Speicherallokierung.

6. Aufgabe

Implementieren Sie eine Funktion `distance`, welche den Abstand sowie den Vektor zwischen zwei gegebenen Punkten A und B im dreidimensionalen Raum berechnet. Einen 3D-Punkt können Sie entweder als Struct oder als Array mit drei Komponenten darstellen. Der Abstand zwischen den Punkten soll als `double` Wert von der Funktion zurückgegeben werden. Der Vektor zwischen den Punkten A und B soll über einen an die Funktion übergebenen Pointer zurückgegeben werden.

Implementieren Sie auch ein Testprogramm.

Hinweis: Verwenden Sie folgende Funktionen aus der Bibliothek `math.h` zur Berechnung der Potenz und der Quadratwurzel: `double pow(double x, double y)` `double sqrt(double x)`

7. Aufgabe

In Programmierung 1 haben Sie bereits das Sieb von Eratosthenes kennen gelernt. Dies ist ein klassischer Algorithmus zum Finden von Primzahlen.

- (a) Implementieren Sie eine Funktion, welche das Sieb von Eratosthenes mit Hilfe von Array-Operationen implementiert.
- (b) Implementieren Sie eine zweite Funktion, welche das Sieb von Eratosthenes mit Hilfe von Pointer-Arithmetik implementiert.
- (c) Implementieren Sie ein Testprogramm, welches Ihre Funktionen testet.